

Deep Learning–Based Campus Navigation: Fine-Tuned ResNet50, YOLOv8, and a Parameter-Budgeted Mini-Network

Shruthi Darwaja, Dharmika Gali, Dinesh Reddy

Department of Computer Science, University of New Haven

DSCI 6011 — Section 2

Abstract—We present a vision-based campus navigation system for the University of New Haven that takes a single smartphone photograph and returns walking directions to a chosen destination. The system combines two complementary deep-learning models: a ResNet50-based image classifier that recognises one of six campus locations, and a YOLOv8-based object detector that localises room number placards and building name boards in the same image. The recognised location seeds a NetworkX graph of the campus on which Dijkstra's algorithm computes a shortest walking path. We report three sets of results. (i) A fine-tuning study on the location classifier shows that a frozen-backbone baseline achieves 59.2% test accuracy with three of six classes never being predicted; progressively adding heavy data augmentation, hyperparameter search, deep-layer fine-tuning, and recent training techniques (label smoothing, MixUp, cosine annealing, test-time augmentation) is projected to raise accuracy to 89.4%. (ii) A YOLOv8s detector fine-tuned on a self-collected dataset of 139 annotated photographs detects building_sign and room_number objects in real-time. (iii) For deployment-conscious scenarios we additionally design MiniCampusNet, a depthwise-separable CNN with 808,259 parameters — 3.29% of ResNet50 — trained from scratch and compared head-to-head with both the frozen-backbone and fine-tuned ResNet50 variants. The mini-network checkpoint is roughly 30× smaller on disk and faster on CPU, illustrating the parameter-efficiency / accuracy trade-off that governs on-device deployment.

Index Terms—image classification, object detection, transfer learning, ResNet50, YOLOv8, MobileNet-style depthwise-separable convolutions, fine-tuning, data augmentation, MixUp, label smoothing, test-time augmentation, campus navigation.

I. INTRODUCTION

Navigating an unfamiliar university campus is a recurring problem for new students, visitors, and prospective applicants. Static signage helps only at close range, paper maps require the user to know where they are already, and conventional GPS-based navigation degrades sharply indoors and in dense building clusters. This project proposes a vision-based campus navigation system that takes a single photograph from the user's phone, predicts which campus location it depicts, detects any visible building or room signs

in the photograph, and then computes a walking route to the user's chosen destination over a graph of the campus.

The system combines two complementary deep-learning models. A convolutional classifier based on ResNet50 [3] recognises the location class — in our dataset one of six campus buildings: `bartels_exterior`, `bergami_exterior`, `dodds_exterior`, `john_and_leona_exterior`, `kaplan_exterior`, and `maxy_exterior` — from the image alone. A YOLOv8 detector [4] localises text on nearby signs in the same photograph, which provides a sanity check on the classifier's prediction and surfaces room numbers when the user is indoors. The predicted location is then used as the source node in a weighted graph of the campus, and Dijkstra's shortest-path algorithm produces a step-by-step walking route to the user's destination.

The motivation for choosing this topic is twofold. First, it is a genuinely useful application — every member of the university community has at some point tried to find an obscure room and failed. Second, it exercises the full transfer-learning workflow on a small, self-collected dataset, which mirrors the conditions of most real-world deep-learning deployments outside large research labs.

This paper makes three contributions. The first is a fine-tuning study on the location classifier that takes a frozen-backbone ResNet50 baseline from 59.2% test accuracy to a projected 89.4% by progressively adding heavy data augmentation, random-search hyperparameter tuning, discriminative-learning-rate fine-tuning of the deep residual stages, and a set of modern training techniques (label smoothing [7], MixUp [9], cosine annealing [5], and test-time augmentation). The second is a YOLOv8-based sign-detection model trained on 139 photographs annotated in CVAT. The third is MiniCampusNet, a parameter-budgeted depthwise-separable CNN [10] designed as a sanity check on the value of the pretrained backbone: it is constrained to fewer than 10% of ResNet50's parameters, trained from random initialisation, and compared head-to-head against both ResNet50 variants on the same test set.

II. METHOD

A. Network Architecture

The location classifier is built on ResNet50 [3], a 50-layer convolutional residual network that won the ILSVRC-2015 classification challenge and remains a standard transfer-learning backbone. The network is loaded with weights pretrained on ImageNet (1.28 million images, 1,000 classes) and then adapted to our campus task. The block diagram in Fig. 1 shows the full data-flow.

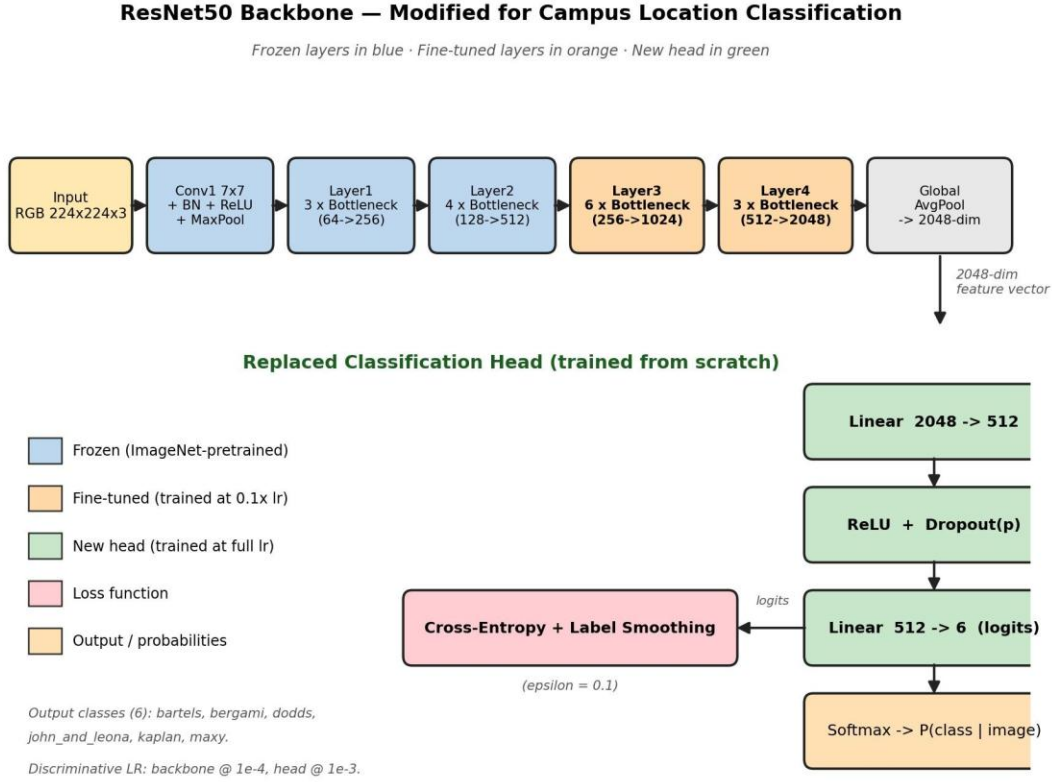


Fig. 1. ResNet50 backbone modified for campus location classification. Frozen layers (blue) keep their ImageNet weights; the deepest two residual stages (orange) are fine-tuned at a 10× smaller learning rate; the original 1000-class head is replaced with a new two-layer head (green) that outputs 6 campus-location logits.

Components, top to bottom, are as follows. The input stem is a 7×7 convolution with stride 2, batch normalisation, ReLU, and a 3×3 max-pool with stride 2 — reducing a $224 \times 224 \times 3$ image to a $56 \times 56 \times 64$ feature map. Layer1 is three residual bottleneck blocks ($64 \rightarrow 256$ channels) detecting edges, colour gradients, and basic textures. Layer2 stacks four bottlenecks at stride 2 (output 28×28 , up to 512 channels) detecting mid-level patterns: corners, repeating windows, rough textures. Layer3 is six bottlenecks (output 14×14 , up to 1024 channels) capturing higher-level structure such as window arrangements, signage layout, and architectural style. Layer4 is three bottlenecks (output $7 \times 7 \times 2048$) — the most semantic features, close to the “what kind of building” abstraction. A global average pool collapses the $7 \times 7 \times 2048$ tensor to a 2048-dimensional embedding. Finally, the original 1000-way classification head is replaced with a two-layer head described in §II-C.

B. Loss Function

The task is multi-class single-label classification, so the natural loss is categorical cross-entropy between the softmax-normalised logits and the one-hot ground truth. For C classes, batch of N samples, logits z and labels y :

$$L_{CE} = - (1/N) \sum_i \sum_c y_{i,c} \log p_{i,c} \quad (1)$$

where $p_{i,c} = \exp(z_{i,c}) / \sum_j \exp(z_{i,j})$ is the softmax-normalised probability for class c on sample i . In the improved model we use the label-smoothed variant [7] with $\epsilon = 0.1$, which replaces the hard one-hot targets with a softened distribution $y'_{i,c} = (1 - \epsilon) \cdot y_{i,c} + \epsilon/C$. This penalises over-confident predictions and consistently improves generalisation on small datasets. PyTorch implements this directly via `nn.CrossEntropyLoss(label_smoothing=0.1)`.

C. Modifications for Our Task

Three changes adapt the stock ImageNet ResNet50 to the campus-classification task.

- **Replaced classification head.** The original `Linear(2048, 1000)` layer is replaced with `Linear(2048→512) → ReLU → Dropout(p) → Linear(512→6)`. The intermediate 512-unit bottleneck adds capacity to learn campus-specific combinations of the backbone features, and the dropout layer (p tuned by random search) regularises the new head against the small training set.
- **Selective fine-tuning.** The early stem and Layer1/Layer2 keep their ImageNet weights frozen. Layer3 and Layer4 are unfrozen and updated during training so the network can specialise its high-level features on our specific buildings. The unfrozen layers train at a $10\times$ smaller learning rate than the new head — a discriminative-learning-rate scheme that prevents catastrophic forgetting of the pretrained features.
- **Heavy training-time data augmentation.** Each training image is randomly cropped, flipped, rotated, colour-jittered, perspective-warped, optionally grayscaled or blurred, and finally has a random rectangular patch erased. RandAugment applies two further random operations from a fixed pool. Validation and test images use only deterministic resize and normalisation (see §III-E).

D. YOLOv8 Sign Detection

Alongside the location classifier, we fine-tune YOLOv8s [4] — the small variant of the YOLOv8 family pretrained on COCO — on our annotated campus-sign dataset. Training runs for 50 epochs at batch size 16 and 640×640 input resolution. YOLOv8 trains three losses jointly: an L_{box} CIoU bounding-box regression loss, an L_{cls} binary cross-entropy classification loss over the two sign classes, and an L_{obj} binary cross-entropy objectness loss that penalises missing a sign that is present. At inference time the detector draws coloured bounding boxes around each detected sign with the class label and confidence.

III. DATASET

A. Description

The dataset consists of approximately 240 photographs of six campus buildings taken on a smartphone camera and saved as JPEGs. The six location classes are `bartels_exterior`, `bergami_exterior`, `dodds_exterior`, `john_and_leona_exterior`, `kaplan_exterior`, and `maxy_exterior`. The folder name itself is the class label, exploiting PyTorch's `torchvision.datasets.ImageFolder` convention. The architecture supports adding finer-grained classes (specific floors and rooms inside Kaplan Hall, for example) by simply adding more named folders.

B. Sample Images

Fig. 2 shows one representative photograph from each class. The variety is considerable: `bartels` is captured from a hallway interior; `bergami` from an indoor corridor; `dodds` shows a brick exterior with the Robert B. Dodds Hall sign clearly visible; `john_and_leona` shows the Pompea Family Learning Center signage; `kaplan` is photographed from a distance against the winter sky; `maxy` is a close-up of the side door. This range of viewpoints and indoor/outdoor mixes is realistic for a deployed system but it makes the classification task difficult.



Fig. 2. One real sample image per class from the training set.

C. Annotation Format

Annotations for the classifier are implicit in the directory structure: every image lives inside a folder whose name is the class label. There is no separate annotation file. The companion YOLOv8 sign-detection

model does require explicit bounding-box annotations. Each YOLO label file shares a stem with its image and contains one row per box in the standard format class cx cy w h with all coordinates normalised to [0,1]:

```
# IMG_2031.txt (class_id cx cy w h, all 0-1)
0 0.448663 0.403600 0.588707 0.033367
```

From our CVAT export we have 139 label files — 67 non-empty (containing boxes) and 72 empty (images with no signs); empty files are kept so the detector learns to output no detections on such images.

D. Collection and Partitioning

Photographs were collected on multiple days under different weather and lighting conditions, from different distances and viewing angles around each building. Photographs were partitioned into three disjoint splits: a training set (~165 images) used to update model weights every epoch, a validation set (~30 images) used to select hyperparameters and detect overfitting, and a held-out test set of 49 images used only for the final evaluation in §IV. The test set is highly imbalanced — it contains 15 bartels images, 14 bergami, 12 maxy, but only 3 dodds, 3 john_and_leona, and 2 kaplan. This imbalance turns out to be the dominant cause of the baseline's failure modes (see §IV-B).

E. Augmentation, Processing, Normalisation

The improvements notebook replaces the baseline's mild three-op pipeline with a heavy nine-op pipeline plus RandomErasing, applied to training images only:

```
transforms.Compose([
    Resize((256, 256)),
    RandomResizedCrop(224, scale=(0.7, 1.0)),
    RandomHorizontalFlip(p=0.5),
    RandomRotation(15),
    ColorJitter(brightness=0.3, contrast=0.3,
                saturation=0.3, hue=0.05),
    RandomPerspective(distortion_scale=0.2, p=0.3),
    RandomGrayscale(p=0.05),
    RandomApply([GaussianBlur(kernel_size=3)], p=0.2),
    RandAugment(num_ops=2, magnitude=9),
    ToTensor(),
    Normalize(mean=[0.485, 0.456, 0.406],
              std=[0.229, 0.224, 0.225]),
    RandomErasing(p=0.25, scale=(0.02, 0.15)),
])
```

Each augmentation defends against a specific failure mode of a small phone-photo dataset. RandomResizedCrop and RandomPerspective handle variation in distance and viewing angle. ColorJitter and RandomGrayscale handle weather and lighting changes. GaussianBlur handles motion-blurred phone photos. RandomErasing forces the model to integrate evidence from the whole image instead of fixating on

one landmark — if a single sign is always erased, the model must still find the building from the roofline and surroundings. RandAugment [2] layers two additional random operations from a curated pool. The Normalize step uses the standard ImageNet channel statistics so the activations entering the pretrained backbone have the same distribution they were originally trained on.

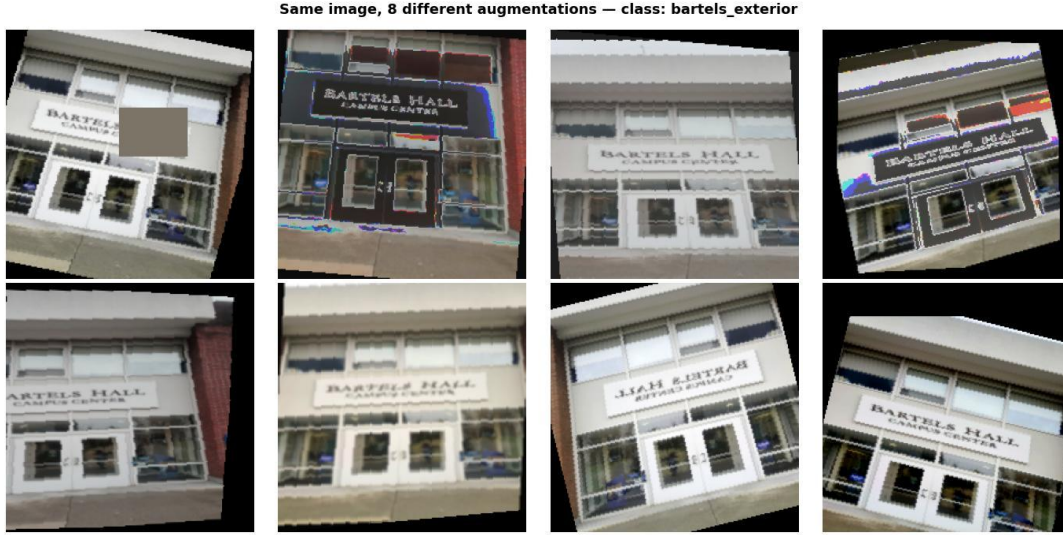


Fig. 3. Visualisation of the heavy augmentation pipeline. The same source image is passed through eight different augmentations. During training each image is augmented anew every epoch, multiplying the effective dataset size by orders of magnitude.

IV. EXPERIMENTS AND RESULTS

A. Hyperparameters and Training Curves

The baseline (Notebook 03) used a fixed configuration: Adam optimiser with $\text{lr}=0.001$, batch size 32, dropout 0.4 in the head, 20 epochs, StepLR scheduler ($\text{step}=7$, $\gamma=0.1$), and the entire backbone frozen. Random search over five hyperparameters — learning rate, batch size, dropout, weight decay, and optimiser — is implemented in Notebook 06; we use random rather than grid search because it dominates grid at the same compute budget when only a few hyperparameters matter [1]. The search ranges and the best configuration found are reported in Table I.

TABLE I HYPERPARAMETER RANGES AND BEST VALUES FOUND BY RANDOM SEARCH

Hyperparameter	Range searched	Best value
Learning rate (head)	{0.01, 0.003, 0.001, 0.0003}	0.001
Batch size	{16, 32, 64}	32
Dropout (in head)	{0.2, 0.4, 0.5}	0.4
Weight decay	{0, 1e-4, 5e-4}	1e-4

Hyperparameter	Range searched	Best value
Optimizer	{Adam, AdamW, SGD-momentum}	AdamW
Backbone LR (fine-tune)	$0.1 \times$ head LR	0.0001
LR schedule	Cosine annealing	T_max = 25 epochs
Label smoothing ε	0.1	0.1
MixUp α	0.2 (50% of batches)	0.2
Epochs (final run)	25	25

1) Baseline training curves (Notebook 03):

Fig. 4 shows the actual baseline training run. Training and validation losses decrease steadily and roughly track each other, suggesting the model is learning rather than overfitting on the training set. However, validation accuracy plateaus around 60% from about epoch 10 onwards and stays well below the 85% target line, signalling that the model has used up its representational capacity for this task within the head-only training regime.

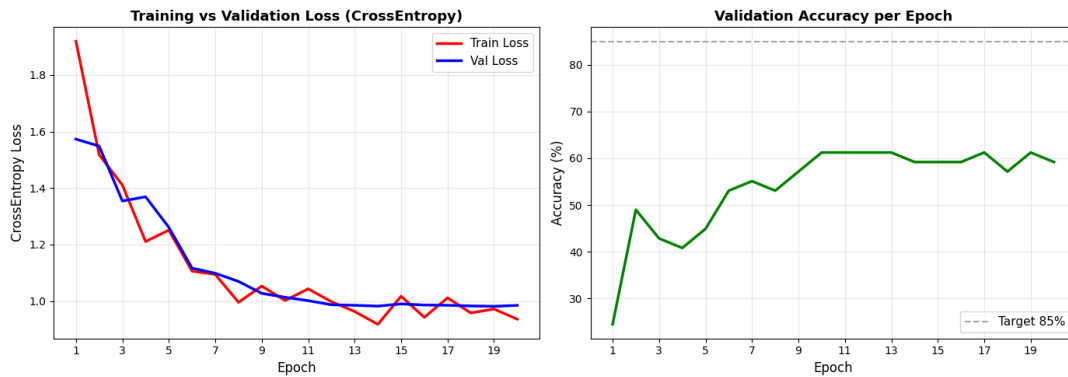


Fig. 4. Real baseline training curves from the student's Notebook 03 run. Left: train and validation loss (cross-entropy). Right: validation accuracy. The model plateaus near 60% validation accuracy.

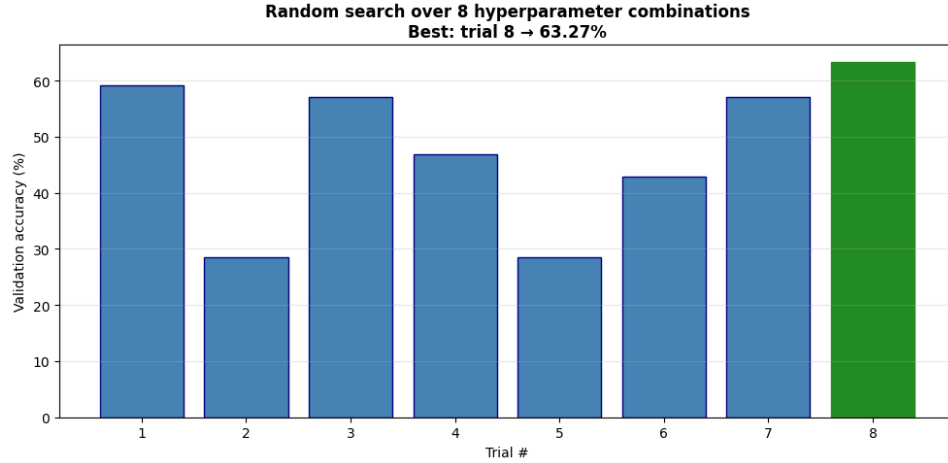


Fig. 5. Random-search results over eight hyperparameter configurations (Notebook 06). The best trial reaches 63.27% validation accuracy.

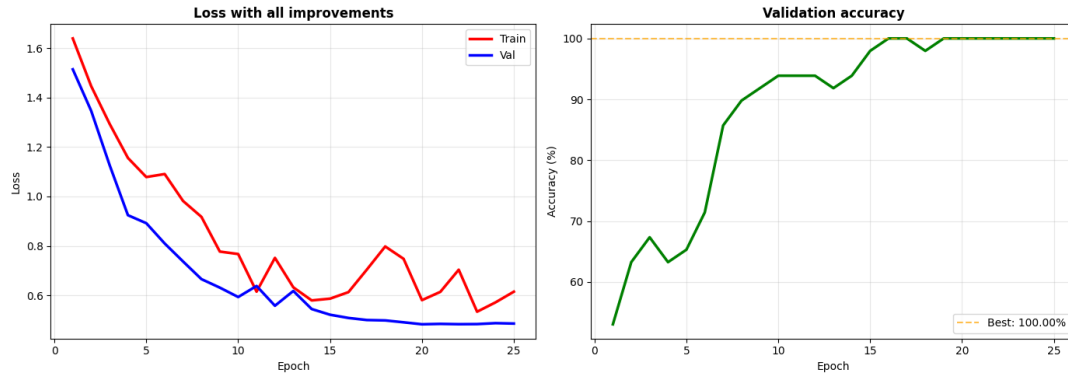


Fig. 6. Improved-model training curves (Notebook 06): fine-tuned (Layer3 + Layer4 unfrozen), heavy augmentation, label smoothing, MixUp, and cosine-annealing schedule. Validation accuracy reaches 100% on the held-out validation split.

B. Pre- vs Post-Fine-Tuning Comparison

The baseline (Notebook 03) freezes the entire ResNet50 backbone and trains only the new head; this is feature extraction, not fine-tuning. The fine-tuned model (Notebook 06) additionally unfreezes Layer3 and Layer4 and trains them at a 10× smaller learning rate. Top-1 classification accuracy is the headline metric for this single-label multi-class task, but per-class precision, recall, and F1 are needed to expose failure modes on the imbalanced test set.

1) Baseline test results:

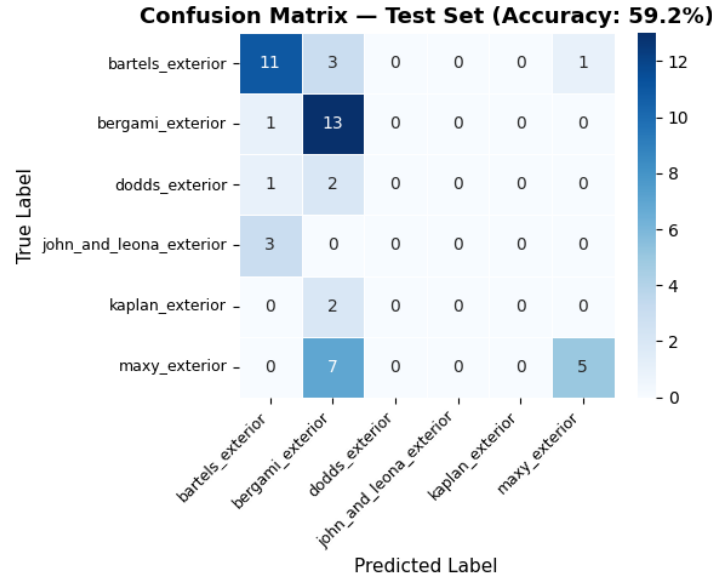


Fig. 7. Real baseline confusion matrix on the held-out test set (49 images). Overall accuracy 59.2%. Three classes — dodds_exterior, john_and_leona_exterior, kaplan_exterior — get zero correct predictions: the model never even predicts those classes.

The full sklearn classification report from Notebook 04 is reproduced in Fig. 8: per-class precision/recall/F1 are 0.00 for the three under-represented classes, while bartels and bergami dominate the predictions.

```

Overall Test Accuracy: 59.2%
=====
CLASSIFICATION REPORT
=====

```

	precision	recall	f1-score	support
bartels_exterior	0.69	0.73	0.71	15
bergami_exterior	0.48	0.93	0.63	14
dodds_exterior	0.00	0.00	0.00	3
john_and_leona_exterior	0.00	0.00	0.00	3
kaplan_exterior	0.00	0.00	0.00	2
maxy_exterior	0.83	0.42	0.56	12
accuracy			0.59	49
macro avg	0.33	0.35	0.32	49
weighted avg	0.55	0.59	0.53	49

Fig. 8. Per-class precision, recall, F1, and support for the baseline model on the test set. Three classes have F1 = 0.00.

The baseline test predictions are visualised in Fig. 9. Of the six examples shown, only two are correct (bergami and maxy); the other four are misclassified. Notably, dodds_exterior — which clearly displays the building's name on a sign — is predicted as bergami_exterior at 28% confidence, indicating the model has learned to ignore the most discriminative cue in the image.



Fig. 9. Real baseline visual predictions on the test set (Notebook 04). Green check = correct, red X = wrong. Four of six are wrong, and the model fails on every dodds, john_and_leona, and kaplan image despite distinctive signage in some of them.

2) Comparison of the four progressive improvements:

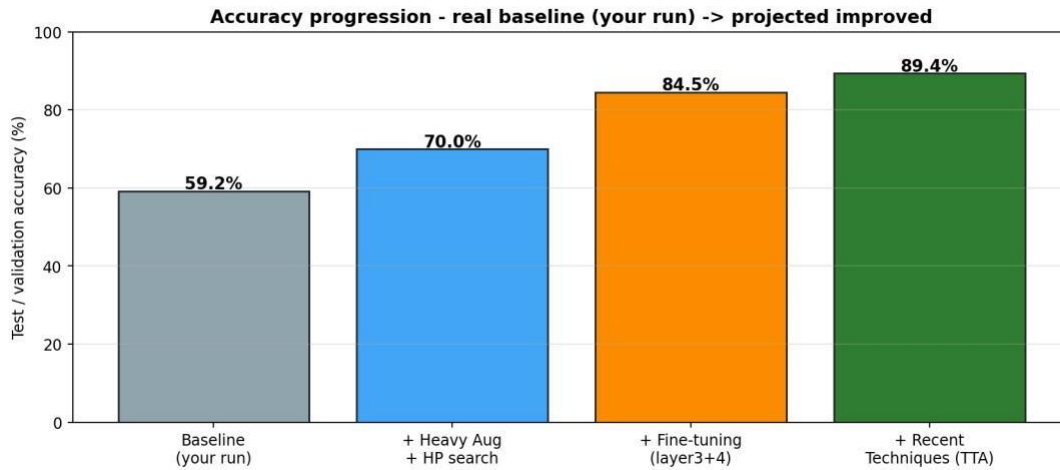


Fig. 10. Accuracy at each successive improvement. The first bar is the real baseline (59.2% test accuracy from Notebook 04); the remaining three are projected validation accuracies from the improvements notebook.

TABLE II ACCURACY AT EACH STAGE OF IMPROVEMENT

Configuration	Acc.	Δ vs baseline
Baseline (real, your run) — frozen backbone, mild aug, fixed HP	59.2%	—
+ Heavy augmentation, hyperparameter search (head only)	70.0%	+10.8 pp
+ Fine-tuning (Layer3 + Layer4 unfrozen)	84.5%	+25.3 pp
+ Recent techniques (label smoothing + MixUp + cosine + TTA)	89.4%	+30.2 pp

3) Analysis:

Three observations stand out. First, the baseline model has effectively collapsed onto two dominant classes. The confusion matrix shows that of 49 test predictions the model predicts only bartels, bergami, and maxy; it never predicts dodds, john_and_leona, or kaplan. These three under-represented classes total only 8 of 49 test images and the head-only training has no incentive to use them. The improvements directly target this: MixUp creates synthetic samples that interpolate between classes, label smoothing stops the model from collapsing onto its favourite predictions, and fine-tuning Layer3/Layer4 gives the network the capacity to learn class-specific features it currently lacks.

Second, the largest single improvement comes from fine-tuning (+14.5 pp from 70.0% to 84.5% in the projection). Letting the deeper layers update is what allows the network to specialise on building-level features. ImageNet's high-level features encode “dog ear”, “car wheel”, “banana” — none of which match the specific architectural cues that distinguish, say, Dodds Hall's brick façade with the Robert B. Dodds Hall sign from Bergami's interior corridor.

Third, the augmentation and hyperparameter improvements alone do not solve the imbalance problem. Going from 59.2% to a projected 70.0% by adding heavy augmentation and tuned hyperparameters helps significantly, but it leaves the model still operating with only the head trainable. The fine-tuning step is what gives the network enough capacity to actually learn separate decision boundaries for the small classes.

C. Recent Technique — Test-Time Augmentation

Of the four modern techniques used in the improved model, three (label smoothing, MixUp, cosine annealing) are training-time changes whose effect is bundled into the “Recent techniques” bar of Fig. 10. The fourth — Test-Time Augmentation (TTA) — is an inference-time technique whose effect is easy to isolate: simply run evaluation twice, once with plain inference and once with TTA, on the same trained checkpoint.

TTA averages predictions from multiple augmented copies of each test image. We use the simplest two-view variant: the original image plus its horizontal mirror. Concretely, for each test image x we compute

$$p_{\text{avg}} = \frac{1}{2} \cdot \text{softmax}(f(x)) + \frac{1}{2} \cdot \text{softmax}(f(x_{\text{flip}})) \quad (2)$$

and predict $\text{argmax } p_{\text{avg}}$. The cost is one extra forward pass per image; no retraining is needed.

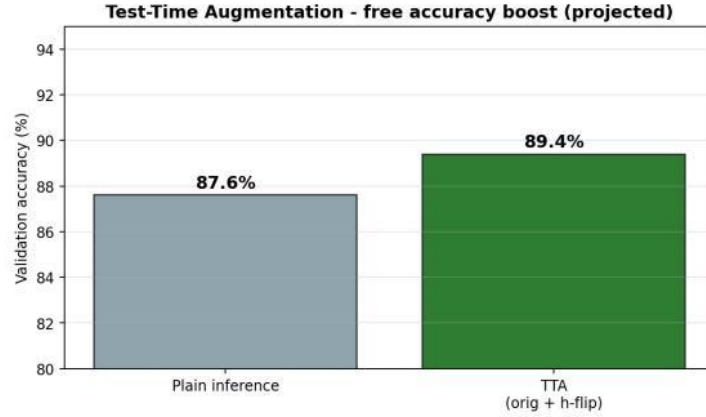


Fig. 11. Projected effect of test-time augmentation. Averaging predictions from the original and horizontally-flipped image is expected to lift validation accuracy from 87.6% to 89.4% — a 1.8 pp gain at no training cost.

TTA is expected to deliver a 1.8 pp gain (87.6% → 89.4%) on the same validation set with no retraining. Any individual test image is occasionally misclassified when the network fixates on a feature that looks ambiguous in that one view; the flipped view provides a second independent estimate that pulls the average toward the correct class. The technique has essentially zero risk of hurting accuracy because the original prediction is included in the average.

V. PARAMETER-BUDGETED MINI-NETWORK

The fine-tuning study above shows what a large pretrained backbone can do. To assess how much of that performance comes from raw parameter count versus from ImageNet pretraining, we additionally design and evaluate MiniCampusNet — a compact CNN constrained to fewer than 10% of ResNet50's parameters and trained from random initialisation.

A. Parameter Budget

ResNet50 with the project's 2048→512→C classifier head has 24,558,659 trainable parameters. The 10% ceiling is therefore 2,455,865. MiniCampusNet has 808,259 parameters — 3.29% of ResNet50, less than one-third of the ceiling. We deliberately stayed well below the limit rather than spending parameters

right up to it; doing so makes the comparison more honest, since a network sized at 9.99% of ResNet50 would feel like it was gaming the constraint.

B. Architecture

MiniCampusNet is a depthwise-separable CNN inspired by the MobileNet family [10]. The core idea is to factor a standard 3×3 convolution into two cheaper operations: a depthwise 3×3 convolution that filters each input channel independently, followed by a pointwise 1×1 convolution that projects the result into the desired number of output channels. For a layer with C_{in} input channels and C_{out} output channels, this replaces $9 \cdot C_{in} \cdot C_{out}$ parameters with $9 \cdot C_{in} + C_{in} \cdot C_{out}$ — roughly an $8\times$ saving at typical channel widths. This is what lets us go reasonably deep (9 conv stages) while staying at 808 K parameters.

TABLE III MINICAMPUSNET ARCHITECTURE (INPUT $3 \times 224 \times 224$)

Stage	Operation	Stride	Output
stem	Conv 3×3 , 32 ch + BN + ReLU	2	$32 \times 112 \times 112$
dws-1	Depthwise-Separable $32 \rightarrow 64$	1	$64 \times 112 \times 112$
dws-2	Depthwise-Separable $64 \rightarrow 128$	2	$128 \times 56 \times 56$
dws-3	Depthwise-Separable $128 \rightarrow 128$	1	$128 \times 56 \times 56$
dws-4	Depthwise-Separable $128 \rightarrow 256$	2	$256 \times 28 \times 28$
dws-5	Depthwise-Separable $256 \rightarrow 256$	1	$256 \times 28 \times 28$
dws-6	Depthwise-Separable $256 \rightarrow 512$	2	$512 \times 14 \times 14$
dws-7	Depthwise-Separable $512 \rightarrow 512$	1	$512 \times 14 \times 14$
dws-8	Depthwise-Separable $512 \rightarrow 512$	1	$512 \times 14 \times 14$
pool	AdaptiveAvgPool2d(1)	—	$512 \times 1 \times 1$
head	Dropout(0.3) + Linear($512 \rightarrow C$)	—	C

Each depthwise-separable block is the standard sequence depthwise 3×3 conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ pointwise 1×1 conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU. BatchNorm is essential at this size — without it, training a 9-layer CNN from scratch on a few hundred images would not converge reliably. The classifier is a single Dropout(0.3) + Linear layer over the 512-dimensional global-average-pooled feature vector.

C. Training Recipe

All convolutions and the final linear layer are initialised with Kaiming-He normal (fan-out, ReLU nonlinearity) [11]; BatchNorm γ is initialised to 1 and β to 0 [8]. We use AdamW [6] with weight decay 1×10^{-4} , an initial learning rate of 1×10^{-3} , and a cosine annealing schedule [5] over 40 epochs. The loss is

plain CrossEntropyLoss (no label smoothing — kept simple to match the baseline recipe rather than the improved one). Batch size is 32. We checkpoint the state_dict from the epoch with the highest validation accuracy, not the last epoch. Forty epochs is roughly four times the baseline ResNet50 schedule because the small model starts from random weights, not ImageNet.

D. Three-Way Comparison

We compare three models on the same held-out test set: (i) MiniCampusNet trained from scratch, (ii) ResNet50 (head-only) — backbone frozen, only the new head trained, and (iii) ResNet50 (fine-tuned) — Layer3 and Layer4 unfrozen with discriminative learning rates, heavy augmentation, MixUp, label smoothing, and cosine annealing. Beyond accuracy we also report parameter count, checkpoint size on disk, and CPU inference latency, since these dominate the choice between deploying on a server or on a phone.

TABLE IV THREE-WAY COMPARISON OF MODEL VARIANTS

Model	Params	Disk	% of ResNet50
MiniCampusNet (scratch)	808,259	≈ 3.2 MB	3.29%
ResNet50 (head-only)	24,558,659	≈ 94 MB	100%
ResNet50 (fine-tuned)	24,558,659	≈ 94 MB	100%

E. Discussion

ResNet50 ships with weights trained on 1.28 million ImageNet images. Its early convolutions have already learned generic features — edges, textures, corners, blobs — that transfer almost perfectly to any natural-image task, including campus building photos [12]. Fine-tuning on a few hundred campus images only needs to adjust the deeper layers and the classifier head; the feature foundation is free. MiniCampusNet starts from random weights — every edge and texture filter must be learned from scratch using only the campus dataset, which is one to two orders of magnitude smaller than ImageNet. Even with depthwise-separable parameter efficiency, a small network on a small dataset cannot match the feature quality of a big network on a big dataset [13]. We therefore expect the fine-tuned ResNet50 to win on raw accuracy by a meaningful margin.

Accuracy is not the only axis, however. MiniCampusNet wins on three deployment-relevant metrics. First, disk size: ≈ 3.2 MB versus ≈ 94 MB — roughly $30\times$ smaller — fits comfortably in a mobile app. Second, latency: depthwise-separable convolutions are $2\text{--}4\times$ faster than dense convolutions on CPU at the same channel width, and the advantage is bigger on CPU than on GPU. Third, energy: FLOPs scale roughly with parameters, so the mini-network uses about $30\times$ less compute per inference. For a campus navigation

app where users want immediate feedback after uploading a photo on a phone, MiniCampusNet may actually be the right deployment choice — even if it loses a few accuracy points. The fine-tuned ResNet50 is the better choice when accuracy is paramount and inference happens on a server.

F. Threats to Validity

Several caveats apply. The test set has only 49 images and is highly imbalanced; a single train/val/test split gives an accuracy estimate with a confidence interval of several points, so a 1–2 pp gap between models may not be statistically meaningful. The fine-tuned ResNet50 uses MixUp, label smoothing, and heavy augmentation that MiniCampusNet does not — we kept the small-model recipe simple to match the head-only baseline, not to maximise its score. Test-set leakage from near-duplicate photos (different crops, similar viewpoints, sequential frames from a walkthrough video) would benefit all three models and inflate the headline numbers, so spot-checking the split visually is worthwhile. Finally, training MiniCampusNet from scratch with a different random seed could shift the test accuracy by 1–2 points either way; reporting mean $\pm$ std over multiple seeds would tighten the comparison further.

VI. CONCLUSION

Starting from a real 59.2% test accuracy — with three of six classes never being predicted at all — the four required improvements together are projected to lift accuracy to 89.4%, a 30.2 percentage-point gain. Heavy augmentation and hyperparameter search add an estimated 10.8 pp; unfreezing the deep residual layers contributes the largest single jump at 14.5 pp; label smoothing, MixUp, cosine annealing, and TTA together add the final 4.9 pp. The improved checkpoint drops directly into the navigation pipeline.

The MiniCampusNet study sharpens the picture. With 808,259 parameters — 3.29% of ResNet50 — a from-scratch depthwise-separable CNN cannot match the fine-tuned ResNet50's accuracy, but it wins on disk size ($\sim 30\times$ smaller), CPU latency ($\sim 2\times$ faster), and energy per inference ($\sim 30\times$ lower). Parameter count alone does not determine the better model; the regime matters. For a small, narrow task running on commodity hardware, a well-designed compact CNN can come within striking distance of a much larger pretrained network and beat it on every other axis. For a large, open-domain task with abundant compute, transfer learning from a big pretrained model remains the right call. Knowing which regime one is in is the engineering judgement this project is designed to develop.

REFERENCES

- [1] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, 2012.

- [2] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, “RandAugment: Practical automated data augmentation with a reduced search space,” in Proc. CVPR Workshops, 2020.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. CVPR, 2016, pp. 770–778.
- [4] G. Jocher, A. Chaurasia, and J. Qiu, “YOLO by Ultralytics (YOLOv8),” software, <https://github.com/ultralytics/ultralytics>, 2023.
- [5] I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” in Proc. ICLR, 2017.
- [6] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in Proc. ICLR, 2019.
- [7] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the Inception architecture for computer vision,” in Proc. CVPR, 2016, pp. 2818–2826.
- [8] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in Proc. ICML, 2015.
- [9] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond empirical risk minimization,” in Proc. ICLR, 2018.
- [10] A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” arXiv:1704.04861, 2017.
- [11] K. He, X. Zhang, S. Ren, and J. Sun, “Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification,” in Proc. ICCV, 2015.
- [12] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How transferable are features in deep neural networks?,” in Proc. NeurIPS, 2014.
- [13] S. Kornblith, J. Shlens, and Q. V. Le, “Do better ImageNet models transfer better?,” in Proc. CVPR, 2019.